Lab Report 1: GPIO

Authors:
J.M. Páez Sandoval, 37945
L.A. Lugo Muñiz, 37915
D. Lopez Moreno, 37984
C.A. Martínez Macías, 33369
Teacher : Carlos A. Villareal
CETYS Universidad
Facultad de Ingeniería
Curso: Microcontroladores y Sistemas Embebidos
Fecha: Abril 27, 2025

Abstract—This report documents the bare-metal implementation of a GPIO-based LED toggle system on the NUCLEO-F411RE development board (STM32F411RE, ARM Cortex-M4). The project was developed in C without HAL or CubeMX, using direct register access through a modular driver architecture. A GPIO driver abstracts all peripheral register operations, while a button module implements edge-detection and software debouncing via the SysTick hardware timer. A single press of the onboard button B1 toggles the state of LED LD2. Results confirmed stable, glitch-free toggle behavior under rapid and sustained button presses.

Index Terms—GPIO, STM32F411RE, bare-metal, embedded systems, button debounce, SysTick, toggle, NUCLEO-F411RE.

I. INTRODUCTION

General-Purpose Input/Output (GPIO) is the most fundamental peripheral in any microcontroller. Every embedded system interacts with the physical world through GPIO pins configured as digital inputs or outputs. Understanding how to control these pins at the register level, without hardware abstraction layers, is foundational for embedded systems engineering.

This practice was carried out on the NUCLEO-F411RE, a development board based on the STM32F411RE microcontroller (ARM Cortex-M4, 16 MHz HSI oscillator). The objective was to configure PA5 as a digital output driving the onboard LED LD2, and PC13 as a digital input reading the onboard button B1, implementing a toggle with click detection and software debouncing.

The entire implementation was written in C with no HAL or CubeMX dependencies, compiled with arm-none-eabi-gcc, and flashed via OpenOCD using the integrated ST-LINK/V2 interface.

II. MATERIALS AND EQUIPMENT

The following hardware and software resources were used:

NUCLEO-F411RE board (STM32F411RE, Cortex-M4)
USB-A to Mini-USB cable (power and ST-LINK programming)
Personal computer, Windows 11
STM32CubeIDE 2.0.0 (toolchain and OpenOCD only)
Visual Studio Code with MSYS2/MinGW64 terminal
arm-none-eabi-gcc cross-compiler
OpenOCD (flashing via ST-LINK/V2)
Custom Makefile build system

III. DEVELOPMENT

A. Project Structure

The project was organized into three layers of responsibility. The GPIO driver handles all direct register access. The button module implements input semantics on top of the driver. The main application orchestrates initialization and the control loop without touching any hardware registers directly.

B. GPIO Driver

The GPIO driver (GPIO_Driver.h / GPIO_Driver.c) maps a C struct directly onto the GPIO peripheral memory layout as defined in the STM32F4 Reference Manual [1]:

```
typedef struct {
  volatile uint32_t MODER;
  volatile uint32_t OTYPER;
  volatile uint32_t OSPEEDR;
  volatile uint32_t PUPDR;
  volatile uint32_t IDR;
  volatile uint32_t ODR;
  volatile uint32_t BSRR;
} GPIO_RegDef_t;
```

Each field aligns with its hardware register at a fixed memory offset. Pointer macros (GPIOA, GPIOC) cast base addresses to this struct type. Functions such as GPIO_PinMode(), GPIO_WritePin(), and GPIO_ReadPin() encapsulate all bit manipulation, accepting the port pointer and pin number as parameters.

C. Button Module

The button module (button.h / button.c) implements edge detection on top of the GPIO driver. Button B1 is connected to PC13 with an external 4.7 kΩ pull-up resistor, so the pin reads logic 1 at rest and logic 0 when pressed.

A falling-edge detection scheme uses a state variable btn_prev storing the previous loop state. A click is registered only on the 1→0 transition, ensuring one event per physical press. Software debouncing rejects events within 20 ms of the previous valid click.

D. SysTick Timer

The SysTick peripheral, built into every ARM Cortex-M4 core [3], was configured to generate an interrupt every 1 ms. The reload register value is:

RVR = (16,000,000 / 1,000) − 1 = 15,999

The SysTick_Handler ISR increments a global ms_ticks counter each interrupt. The debounce filter compares the current ms_ticks value against the timestamp of the last valid click, rejecting any event arriving within the 20 ms window.

E. System Flowchart

Fig. 2 presents the complete program flow. Initialization (SysTick, GPIO PA5 output, Button_Init) runs once. The main loop polls Button_WasClicked() continuously; a confirmed edge toggles the LED state and updates the physical pin. The SysTick ISR runs asynchronously in the background and the main loop never blocks.

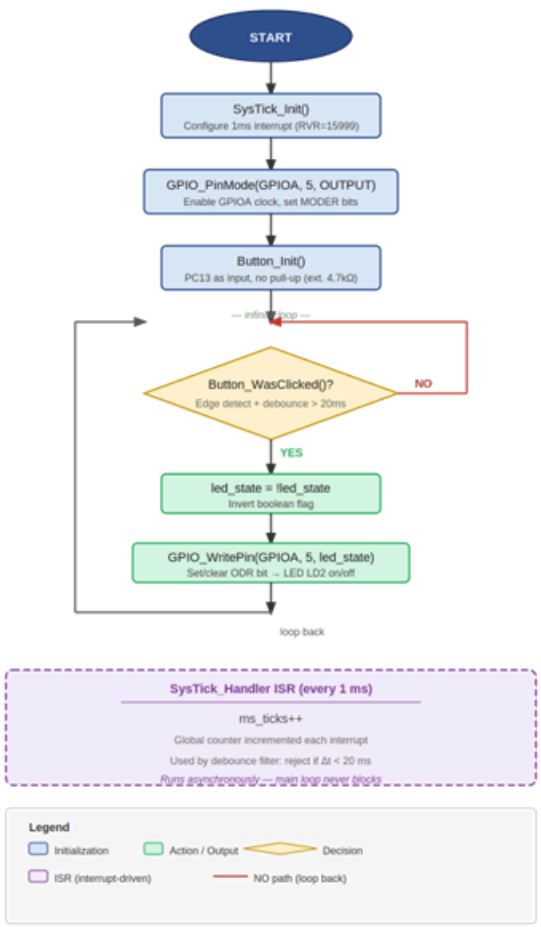

Fig. 1: Program flowchart of the GPIO LED toggle application on STM32F411RE

F. Main Application

The main application initializes SysTick, configures PA5 as output, calls Button_Init(), and enters an infinite loop. On each iteration, Button_WasClicked() is evaluated. A valid click toggles the led_state variable and updates the physical pin via GPIO_WritePin(). The loop never blocks; all timing is handled by the hardware timer.

IV. RESULTS

Once the firmware was deployed to the board, the system operated correctly from the first test. Each time the user button was pressed, the green LED responded by switching between its on and off states, validating that both the input driver and the output control logic had been implemented properly.

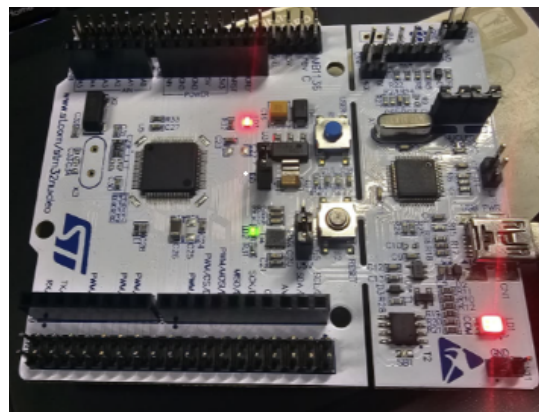

Fig. 2: STM32 Nucleo-F411RE running the toggle LED application

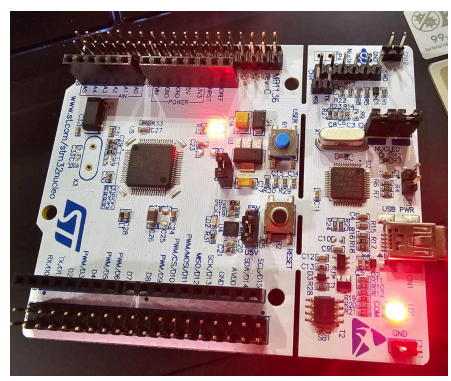

Fig. 3: STM32 Nucleo-F411RE with LED turned on

The digital input configuration on pin PC13 followed an active-low scheme, where a LOW signal on the line indicates a button press and a HIGH signal indicates its release. This translation was managed entirely within the driver layer, keeping the main loop clean and free of any hardware-specific logic.

Throughout testing, the LED toggled consistently with no missed inputs or erratic behavior observed. Every

button press produced exactly one state change in the output, confirming that the communication between input detection and LED control was stable and reliable. The final system met all the expected functional requirements of the assignment.

Two technical obstacles arose during the development process. The first involved a recurring issue when attempting to program the board, where the debugging tool was unable to establish communication with the target device. This was traced back to a previously loaded program that had repurposed the pins used by the debug interface, blocking the programmer from taking control. The issue was resolved by holding the hardware reset while initiating the flash sequence, which prevented the conflicting code from executing and allowed the programmer to take over successfully.

The second obstacle was related to mechanical noise in the button signal. When pressed, the physical contacts do not settle immediately, causing the signal line to oscillate rapidly for a brief period. This led the microcontroller to register multiple events from a single press, resulting in inconsistent LED behavior. A short software delay introduced after each detected press gave the signal time to stabilize, eliminating the false triggers and making the button response predictable and accurate.

V. CONCLUSION

The completion of this lab successfully validated the use of bare-metal GPIO control on the STM32F411RE microcontroller. Operating directly at the register level offered a thorough understanding of key low-level operations such as clock activation, pin mode assignment, and digital input reading — all of which are typically hidden behind HAL abstraction layers.

The decision to structure the project around a modular driver architecture proved effective in keeping hardware-specific code separate from the application logic. Each component fulfilled a distinct role: the GPIO driver contained all direct register interactions, the button module managed input interpretation, and the main file coordinated the overall system behavior. This separation not only improves readability but also makes future modifications significantly easier.

The integration of the SysTick timer introduced a hardware interrupt-driven approach to timekeeping, enabling non-blocking time measurement that is well suited for real-time embedded applications. Overall, this practice reinforced the value of building a solid understanding of the underlying hardware before depending on higher-level software abstractions.

REFERENCES

[1] STMicroelectronics, “STM32F411xC/E Reference Manual (RM0383),” Rev. 3, 2019.
[2] STMicroelectronics, “NUCLEO-F411RE User Manual (UM1724),” Rev. 14, 2021.
[3] ARM Limited, “Cortex-M4 Devices Generic User Guide (DUI0553),” 2010.
[4] J. Valvano, Embedded Systems: Introduction to ARM Cortex-M Microcontrollers, 5th ed., 2014.